Hermetic Knowledge Isolation (HKI)

Executive Brief

By Henok Ghebrechristos, PhD

> “
>
> ***Audience.*** *Engineering leaders, enterprise architects, security leaders, AI platform owners, and technical decision-makers.*
>
> ***Reading time.*** *~5 minutes.*

In One Sentence

Hermetic Knowledge Isolation (HKI) is a runtime contract for enterprise agentic systems: every artifact belongs to one domain, every runtime operation runs in one active domain, and multi-domain business questions are composed through scoped delegation and explicit release, never through silent global fallback.

Put in current data-governance language: HKI operationalizes inference-time data sovereignty. It gives enterprises usage control, provenance, and auditable information-flow boundaries after access has already been granted.

Why This Matters Now

Enterprise AI platforms are moving beyond simple search. They now rewrite queries, traverse graphs, call tools, cache results, run background workflows, and synthesize answers from many intermediate steps.

That makes isolation failures easier to create and harder to notice.

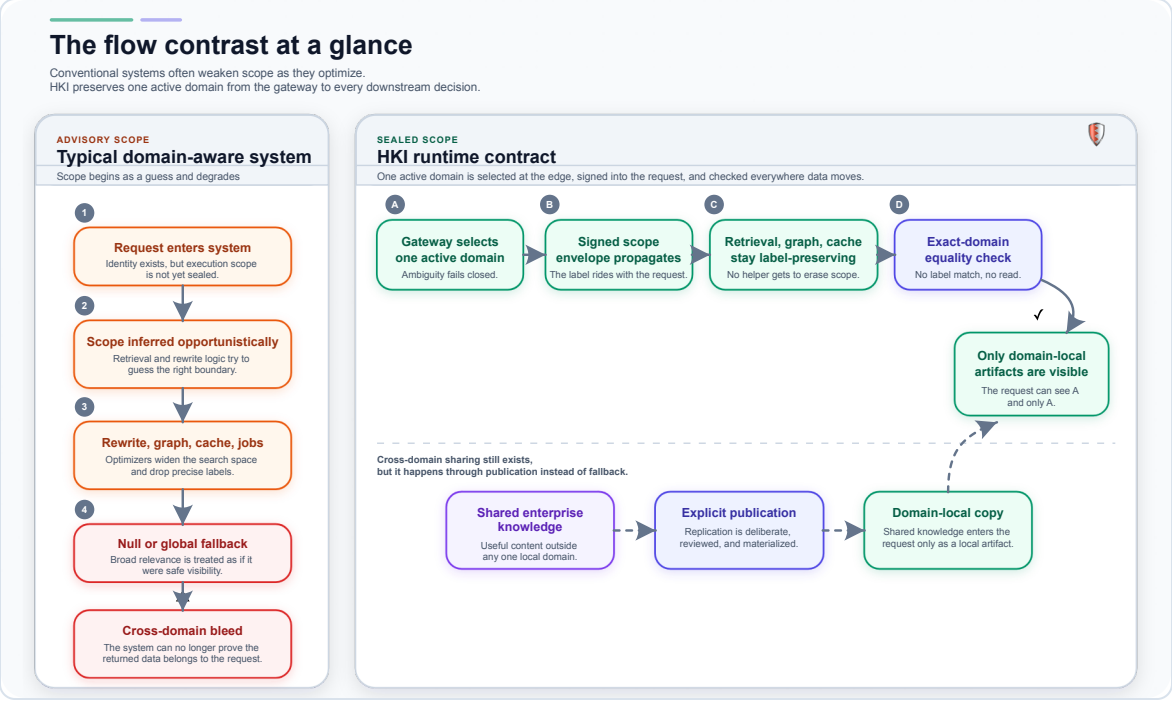
In many systems, the user interface looks scoped while the runtime underneath is not. Scope begins as a query parameter, then weakens as the request moves through rewriters, caches, graph traversal, background jobs, or operator shortcuts. The result is not always a dramatic breach. More often, it is a quiet wrong answer sourced from the wrong business boundary.

HKI is designed to stop that class of failure.

The test is simple: if a runtime path can answer without an explicit active domain, reuse a cache hit from another domain, or treat null-scoped knowledge as globally visible, it is not HKI-conformant.

The Picture

The contrast is straightforward: conventional "domain-aware" systems often treat scope as advisory, while HKI treats it as a sealed execution label.



The Contract

HKI asks teams to adopt four rules.

1. **Every artifact has one domain.** Documents, chunks, graph nodes, review records, jobs, caches, and derived outputs must all be labeled.
2. **Every runtime operation has one active domain.** A user may be authorized for many domains, and a business request may delegate across several of them, but each retrieval, tool, cache, memory, and synthesis step still runs in exactly one active domain.
3. **Shared knowledge is published, not globally visible.** If the same content must appear in several domains, the system materializes domain-local copies explicitly.
4. **Ambiguity fails closed.** Missing scope, contradictory scope, null scope, or unauthorized scope causes rejection, not fallback.

If you remember only one rule, remember this one:

> *Runtime visibility requires exact-domain equality.*

In shorthand: a request running in Domain A must not observe artifacts labeled Domain B unless the system has explicitly published that knowledge into A.

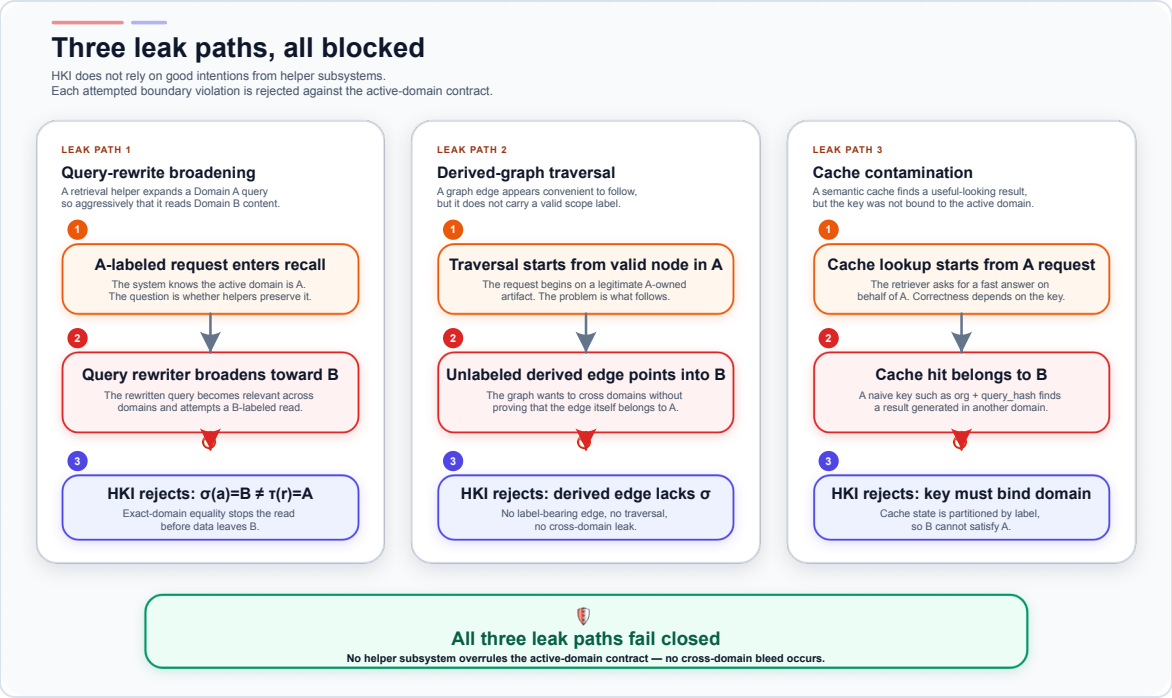
Multi-domain business questions are still supported. For an executive legal-plus-pharmacy briefing, the orchestrator fans out to legal and pharmacy child envelopes, releases approved summaries into an executive briefing domain, and synthesizes the final answer there. The forbidden shortcut is a single mixed prompt or cache path that reads several domains by treating broad `authorized_domains` as runtime read visibility.

The minimum proof points are concrete:

1. Every runtime endpoint requires a non-global active domain.
2. Every persisted runtime artifact carries exactly one domain.
3. Caches include the active domain in their keys.
4. Graph traversals reject unlabeled or different-domain edges.
5. Jobs and review records preserve domain through every stage.
6. Readiness checks fail when null-scope runtime artifacts are found.

What HKI Blocks

HKI is useful because modern leak paths do not come only from one missing database filter. They emerge from intermediate steps.



The three recurring failures are:

1. **Query-rewrite broadening.** A recall optimizer widens a query beyond the caller's domain.
2. **Derived graph bleed.** A graph edge or derived node crosses boundaries because the extraction pipeline failed to preserve labels.
3. **Cache contamination.** A cache key omits the active domain and replays results from another domain.

HKI blocks all three for the same reason: the system must preserve the active domain through every transformation, not only at the final read.

What Changes in Architecture

HKI changes two architectural habits.

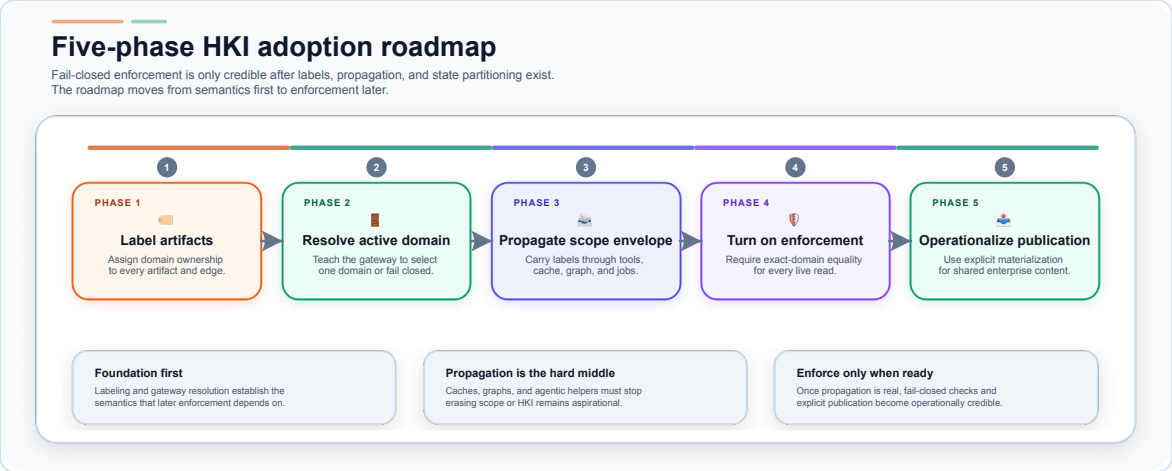
First, it splits the platform into a **runtime plane** and an **admin plane**.

- The **runtime plane** is hermetic. It handles live retrieval, chat, tools, ingestion, review, and release workflows inside one active domain per operation.
- The **admin plane** may inspect across domains, but only through dedicated audit and oversight surfaces. Cross-domain visibility is never smuggled back into runtime endpoints.

Second, HKI replaces implicit shared visibility with **explicit publication**. Shared enterprise policies, playbooks, or curated artifacts are copied into domains through a publication workflow rather than left unscoped.

What Teams Would Actually Do

No enterprise adopts HKI in one cutover. A practical rollout is phased.



The near-term sequence is:

1. **Inventory and audit.** Find null-scoped artifacts, unlabeled graph structures, weak cache keys, and endpoints that can answer without an explicit domain.
2. **Normalize labels.** Ensure every artifact type carries exactly one domain identity.
3. **Harden propagation.** Select and sign the active domain at the edge; reject missing or contradictory scope in downstream services.
4. **Enforce fail-closed behavior.** Turn null-scope and non-preserving paths into readiness failures.
5. **Operationalize it.** Add observability, adversarial tests, and periodic audits that prove the boundary still holds.

The adversarial tests should be blunt: missing scope is rejected, unauthorized active domains are rejected, cross-domain graph edges are not traversed, cache keys cannot omit domain, and publication creates new domain-labeled copies rather than exposing a shared global object.

Costs and Tradeoffs

HKI is not free.

- It adds workflow complexity because publication and replication must be explicit.
- It can increase storage because some shared content is materialized per domain.
- It makes ambiguous user flows stricter because the system rejects rather than guesses.
- It forces teams to clean up old shortcuts, especially in caches, background jobs, and admin endpoints.

Those costs are real. The payoff is equally real: isolation stops being a convention and becomes a runtime contract.

Why It Is Relevant

HKI does not claim novelty at the level of security labels. The underlying ideas are familiar.

It is relevant because enterprise agentic systems now fail in places that older access-control designs did not fully model: graph traversal, query rewriting, semantic caches, long-running workflows, and tool execution.

It complements row-level security, tenant isolation, Zanzibar-style authorization, vector metadata filters, and DLP controls. Those mechanisms answer important local questions. HKI adds the whole-runtime question: did every transformation preserve the one active domain selected for this operation?

HKI gives those systems a single, testable rule for runtime observation:

> *one operation, one active domain, no implicit global visibility.*

Bottom Line

HKI is a practical answer to a modern enterprise AI problem.

It says that semantic boundaries are real only if they survive the entire runtime path: retrieval, graph traversal, caches, tools, jobs, and publication workflows.

That is the value of the model. It turns "domain-aware" from a claim into an enforceable execution contract.